

总数不超过 320，并引入 2.0 秒的结构分析软预算检测点，若超时则主动退出深层探索并安全回退）。此类工程保护机制旨在规避极端复杂拓扑下的退化搜索，并不构成整体端到端求解时间的渐近复杂度保证；

3. **离散事件模拟评测开销：** 单次离散事件评测耗时 T_j 取决于评测内核在推进离散事件时的队列操作与硬件模拟规模。本文各求解器将单用例的在线评测调用次数 b 限制在固定常数上限内（问题一至多 10 次，问题二与问题三至多 3 次），避免了评测次数随图规模无限制累加。结合前置结构校验与分支守卫，实测端到端求解耗时（具体数值见第 7 章）处于工程实用范围之内；赛题第 5 页脚注 1 所提的 5~10 分钟属于避免暴力迭代的效率建议，并非算法运行耗时的下限指标。

4 问题一：独立任务执行下的多核调度

4.1 问题分析与时序约束

在问题一的系统设定中，每个非空子图 $s \in \mathcal{S}$ 均被底层编译系统独立封装为一个物理硬件任务（Task）。由于子图之间在硬件执行层面上严格隔离，芯片私有缓存无法实现跨 Task 的数据保留。无论两个具有数据依赖关系的 Task 最终被指派至同一个计算核心还是不同的计算核心，它们之间传递的中间数据张量均必须通过外部主存（DDR）进行持久化中转，即由前驱 Task 将生成张量执行 COPY_OUT 写出至 DDR，再由后继 Task 执行 COPY_IN 从 DDR 读回片上缓存。

设 Task s 在指派核心 $a(s)$ 上的开始执行时刻与完成时刻分别为 A_s 和 F_s （单位为 cycle）。记 $\text{prev}(s)$ 为排定在同一核心上紧邻 s 之前执行的 Task， $\text{pred}(s)$ 为在数据依赖关系上先于 s 执行的全部前驱 Task 集合。底层离散事件评测系统对硬件调度时序施加了如下物理约束：

$$A_s \geq F_{\text{prev}(s)} + \Delta_{\text{local}}, \quad \Delta_{\text{local}} = 100 \text{ cycles}, \quad (4.1)$$

$$A_s \geq F_t + \Delta_{\text{sync}}, \quad \forall t \in \text{pred}(s) \text{ 且 } a(t) \neq a(s), \quad \Delta_{\text{sync}} = 1000 \text{ cycles}. \quad (4.2)$$

其中， $\Delta_{\text{local}} = 100 \text{ cycles}$ 为同一核心上相邻任务间的上下文切换开销； $\Delta_{\text{sync}} = 1000 \text{ cycles}$ 为跨核心依赖同步屏障引入的硬件等待时延。若某项约束中的前驱对象不存在，则对应不等式不生效。

上述时序约束揭示了多核心切分与调度决策中的基本权衡关系（trade-off）：

1. **并行收益与排队削减：** 将互无计算依赖的算子划分至不同的子图并分配给多个计算核心，能够让无依赖的算子在物理分离的流水线（PIPE_M 与 PIPE_V）上并发推进。虽然这并不改变由原始数据依赖所决定的固有关键路径长度，但能够缩短由于单核执行资源有限而被强制串行化的排队等待时间；
2. **通信开销与同步代价：** 若子图切分引入了跨子图的数据流动边界，中间数据张量必须通过 DDR 进行读写传递，从而产生额外的数据搬运开销与总线争用；当数据依赖跨越不同核心时，更会触发长达 1000 cycles 的跨核同步屏障等待。此外，若多个核心并发访问 DDR，还可能引发内存总线带宽争用，增加各 Task 的实际等待时间。

因此，多核心切分并非越细越好。只有当并发执行所节省的排队等待周期足以弥补切分所引入的 DDR 搬运与跨核同步开销时，切分方案才能实现全局执行时间的净收益。

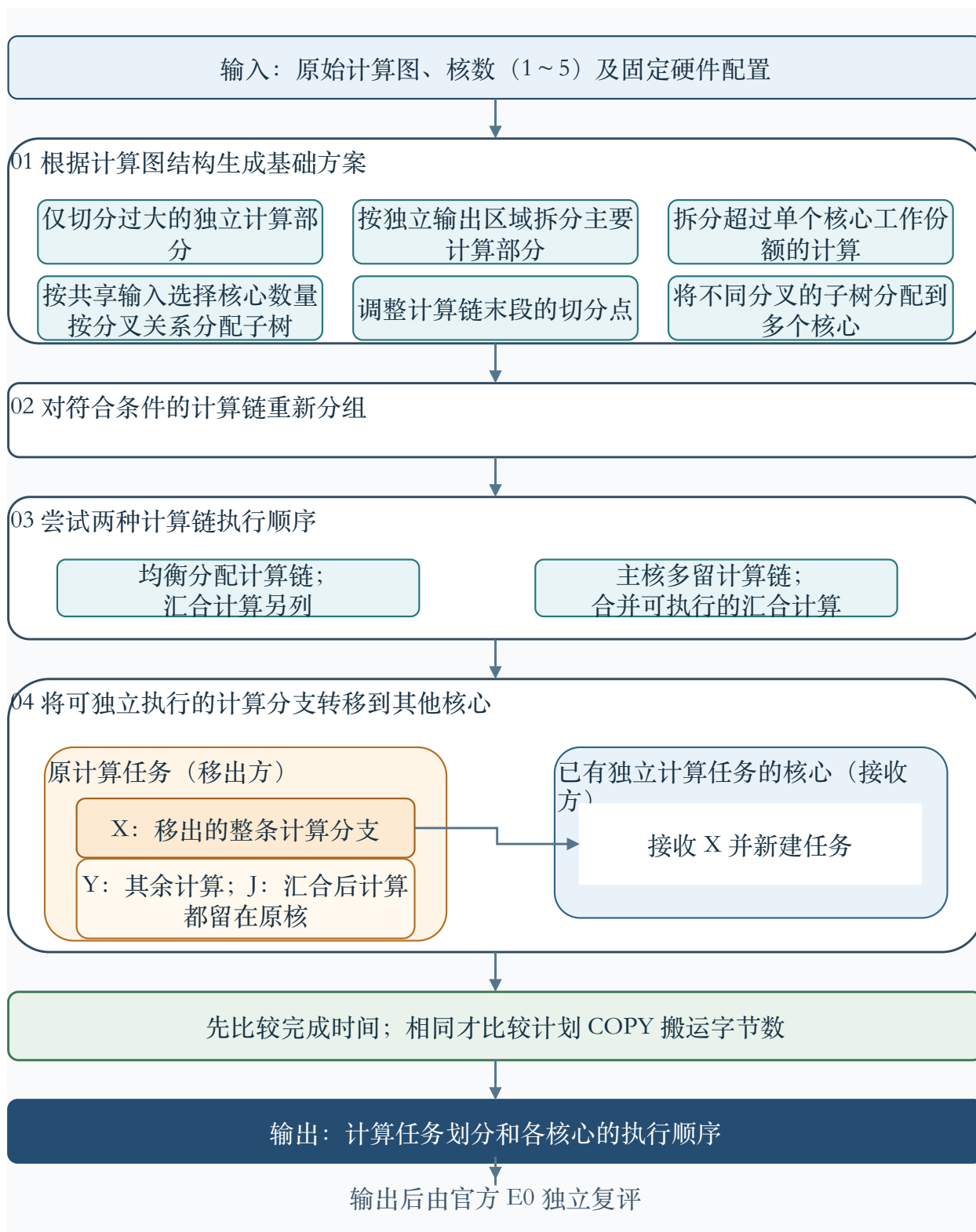


图 4.2-1 问题一固定算法的全流程构造与评测选优逻辑。展示六类基础候选生成、两级细化阶段的在线 E1 评测预算控制与逐阶段择优规则。若分支未产生新结构、格式校验未通过或评测预算耗尽，均自动保留已有合法最优方案，全流程评测调用上限为 10 次，最终方案由 E0 独立复评验证。

4.2 结构化候选方案的构造

4.2.1 基础候选方案

基于上述物理权衡，算法首先分析输入计算图的数据流动特征。若仅考虑算子之间的直接与间接计算依赖，能够沿无向连通边相互到达的算子子集构成了一个弱连通分量 (weakly connected component)。不同连通分量之间天然不存在计算依赖，但在读取共享外部输入时仍可能产生总线访问争用。

针对该计算图结构，算法参考前序研发建立的分叉阶段与前沿子树划分策略，在统一求解入口 (unified.py) 中设计了六类互补的基础结构候选生成器。针对给定用例与核心数 K ，求解器至多构造六种不同侧重点的基础方案：

1. **连通分量前沿划分与大任务重组 (bounded)**：首先调用树状前沿划分模块 (tree_frontend.py, 参数 packet_factor=4) 生成初始划分；随后对操作数超过 4096 的巨型任务进行内部扫描，在不切断任何依赖链的前提下识别其内部独立的计算连通分量，按目标约 1024 个算子的粒度安全重组为独立子图，作为基础参考方案；
2. **大工作量后缀波次剥离 (heavy-or-sink)**：当单个主要分量在主导流水线上的计算量占比达到或超过 80% 且分量数充足时，将该瓶颈分量的末端汇点算子剥离为粗粒度后缀波次 (若不满足触发条件则回退至分层剥离汇点算子的 sink_peel 候选)；
3. **过载连通分量切分 (overload)**：针对单个连通分量在任一流水线上的计算工作量超过全系统平均核心公摊配额的情形，将其剥离出的独立算子包按负载均衡启发式分配至轻载核心，摊薄瓶颈核心的计算压力；
4. **共享输入开销感知划分 (shared-input)**：当连通分量数不小于核心数且外部输入张量总大小超过 $524,288 \text{ B} = 512 \text{ KiB}$ 时，统计各分量对外部输入数据的共享重叠度，将重度共享同一张量的分量优先归并至同一核心，以抑制重复 DDR 搬运总量；
5. **前沿子树划分与并行展开 (fork-frontier)**：采用前序研发建立的分叉阶段与前沿子树划分策略，并推广至无分叉链及内向树 (in-trees) 结构，沿拓扑阶段以前沿粒度 (grain=4) 将不相交的子树打包分配至各核心，挖掘拓扑天然存在的横向并行性；
6. **受限链状结构私有化 (capacity-return)**：严格识别私有的矩阵-向量链-矩阵模式 ($M \rightarrow V^+ \rightarrow M$ 且矩阵算子数恰为分量数 2 倍)，在片上缓存容量满足保守估算的前提下进行局部紧凑分组，规避中间向量张量的外存溢出。

图 4.2-2 给出了一个三核心 ($K = 3$) 场景下的典型分叉汇聚 (fork-join) 计算图的任务切分与分配构型示意。该图仅用于阐释多核心任务切分、核心分配以及执行次序的形式化拓扑关系，并非底层的真实执行时间轴，亦不代表特定用例的独立支路迁移候选。计算图由源头计算 $s \rightarrow a$ 出发，分叉为三条互无计算依赖的并行支路：支路一 ($b_1 \rightarrow c_1 \rightarrow d_1$)、支路二 ($b_2 \rightarrow d_2$) 与支路三 ($b_3 \rightarrow c_3 \rightarrow d_3$)，最终汇总至汇聚计算 $r \rightarrow t$ 。系统将其划分为 5 个任务并分配给 3 个计算核心：

- 核心 0 负责 Task 0 (包含 s, a) 与 Task 1 (包含 b_1, c_1, d_1)；
- 核心 1 负责 Task 2 (包含 b_2, d_2) 与 Task 4 (包含 r, t)；
- 核心 2 负责 Task 3 (包含 b_3, c_3, d_3)。图中实线有向边表示指派在同一核心内部的任务执行先后次序，虚线有向边表示跨越不同核心的任务间数据依赖。在问题一的系统规则下，所有任务边界之间的数据流转均须通过外部主存 (DDR) 中转，跨核心依赖还会引入跨核硬件同步时延。

4.2.2 多级细化与在线择优

在上述基础方案的基础上，求解器通过两级进阶细化模块展开多级候选探索：

- **动态分组长度细化与输入错峰保序 (structural_refine.py)**：在基础候选择优结果的基础上，一方面对链状流水结构进行分组长度调整 (至多产生 1 次在线评测)；另一方面

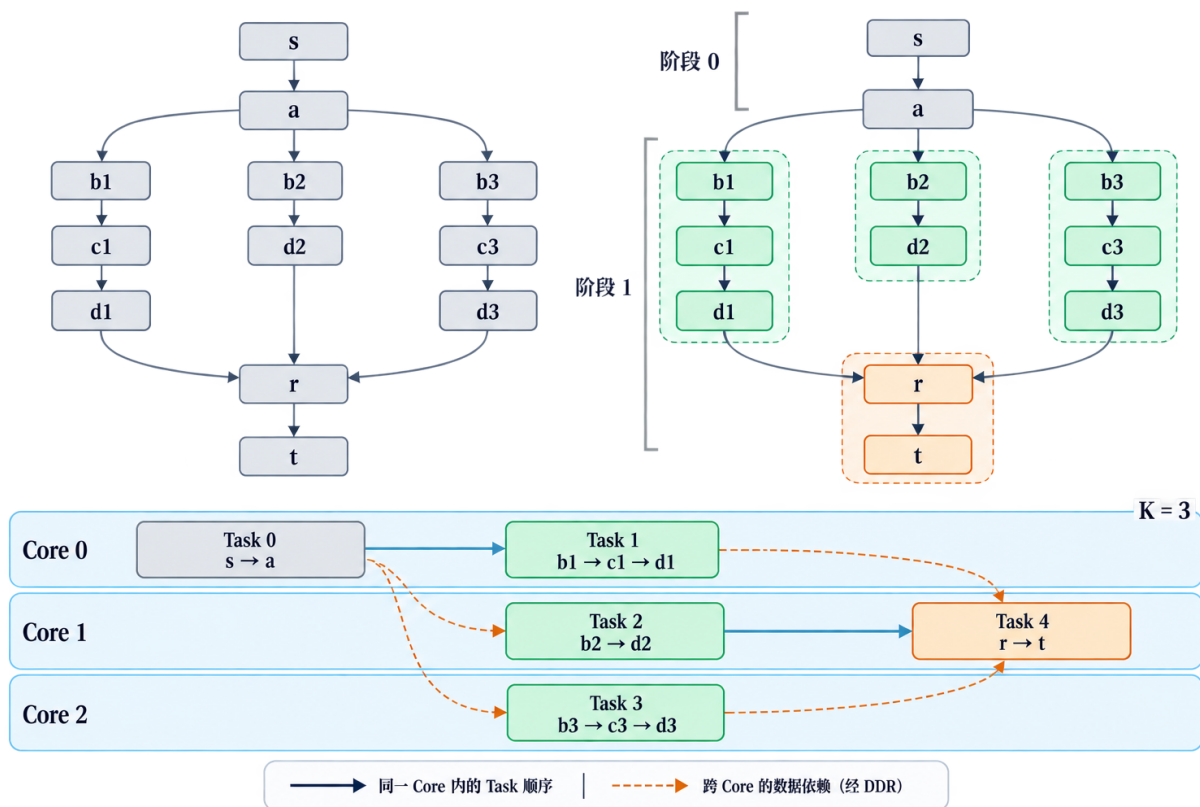


图 4.2-2 三核心场景下的分叉汇聚结构任务划分与多核分配示意。包含 5 个 Task 与 3 个计算核心（核心 0 处理 Task 0 与 Task 1，核心 1 处理 Task 2 与 Task 4，核心 2 处理 Task 3）。实线有向边表示同一核心内部的执行全序，虚线有向边表示跨核心的任务数据依赖，所有 Task 边界的数据流动均通过外部主存（DDR）中转。

- 尝试 **paced** 与 **root-heavy-fused** 两种满足依赖的执行次序（至多产生 2 次在线评测；输入错峰候选的单例时间线见附录 D 图D-1），从中选出性能最佳的父级方案 P_{parent} ；
- **分支援助重分配 (branch_aid.py & branch_refine.py)**：针对父级方案中各核心在同一拓扑阶段负载失衡的情形，通过桥树分析识别可转移的独立计算分支，将其重分配至同波次已有任务的轻载援助核心（至多产生 1 次在线评测）。需要指出的是，援助核心分配采用轻载贪心启发式，并不保证单步消除所有依赖环路，依赖外围拓扑合法性检验进行闭环保护。

全流程采用模拟器 E1 进行在线评分与方案筛选，且每一阶段均执行方案对象比对去重与格式合法性校验。未产生新结构或与已有方案完全重复的候选不参与评测；各阶段若未产生新方案、校验未通过或预算耗尽，算法均安全保留已有合法最优方案，全流程在线 E1 评测调用次数限制在 10 次以内。整体多候选协同生成、在线评测预算控制与逐阶段择优控制流如图 4.2-1 所示。

算法 4.1 问题一多候选协同生成与评测选优算法

```

1 # 输入：原始计算图 graph，核心数 cores (1..5)
2 # 输出：选中的多核心调度方案 plan (预算 6+1+2+1 <= 10 次)
3 parent_plan, parent = structural_refine.solve(graph, cores)
4 if cores < 2 or not valid_parent_evidence(parent, parent_plan):
5     return parent_plan # 单核或无合法父证据直接返回

```

```

6
7 # 尝试构造分支援助方案并校验（伪代码抽象：结构不支持、软超时或异常返回
  None）
8 candidate = safe_construct_branch_aid(graph, cores, parent_plan)
9 if candidate is None or plan_bytes(candidate) == plan_bytes(
  parent_plan):
10     return parent_plan # 重复方案或构造失败直接返回
11
12 # 至多 1 次在线 E1 评测，按字典序 (Makespan, scheduled_copy_bytes) 择
  优
13 cand_obj = safe_score_e1(graph, candidate) # 评测校验并解析目标，异常
  或失败返回 None
14 try:
15     if cand_obj and cand_obj < objective_pair(parent.get("
  selected_objective")):
16         return candidate # 严格字典序改善才采纳
17 except Exception:
18     pass
19 return parent_plan

```

源码对应：834d8c9, src/q1/branch_refine.py::solve 与 unified.py（其中 safe_construct_branch_aid 与 safe_score_e1 为伪代码抽象）。前者封装了 branch_aid.construct 的 2.0s 软超时控制、结构守卫与候选校验；后者封装了单次在线 E1 评测、状态校验与 (makespan, scheduled_copy_bytes) 目标解析（异常或非 ok 返回 None）。全流程在线评测由基础候选（≤ 6）、动态分组（≤ 1）、保序（≤ 2）与分支援助（≤ 1）共享至多 10 次预算，候选去重后按严格字典序 (M_1 , scheduled_copy_bytes) 改善才采纳。

4.3 基于支路重分配的调度优化

为了在不打破既有调度结构的前提下进一步压低瓶颈核心的执行延迟，本文设计了分支援助重分配机制（Branch-Aid）。该机制以 structural_refine 输出的优秀方案作为父级解 $P_{\text{parent}} = (\pi_0, a_0, \sigma_0)$ ，仅在多核心场景（ $2 \leq K \leq 5$ ）下尝试对同阶段核心负载进行横向均衡。以典型算例 case_026（5 核）为例，图 4.3-1 展示了将瓶颈核心独立分支重分配至轻载援助核心前后的切分结构映射（仅作为定性局部结构示意，不代表全局图构型）。

4.3.1 拓扑波次与支路识别

算法将父方案中的全部 COPY 节点暂时收缩，构建关于 Task 顶点的有向依赖图 $G_{\text{task}} = (\mathcal{S}_0, E_{\mathcal{S}_0})$ 。对于每个 Task $s \in \mathcal{S}_0$ ，定义其拓扑高度（height）为从任意源点 Task 出发到达 s 的最长依赖路径边数：

$$h(s) = \begin{cases} 0, & \text{若 } \text{pred}(s) = \emptyset, \\ 1 + \max_{t \in \text{pred}(s)} h(t), & \text{若 } \text{pred}(s) \neq \emptyset. \end{cases} \quad (4.3)$$

算法要求父方案必须严格满足**波次单调推进性质**：在每一个核心 k 的执行序列 $\sigma_{0,k}$ 中，Task 的执行次序必须沿高度 h 严格单调递增，即若 Task s 排定在 Task t 之前，则必有 $h(s) < h(t)$ 。因此，在任意固定高度 h 处，每个核心至多分配有一个 Task。将高度为 h 的全部 Task 构成的集合记为波次 W_h 。

算法遍历每个波次 W_h ，并检查以下强约束条件：

1. **同波次无依赖条件**: 在波次 W_h 内部, 来自不同核心的 Task 之间不得存在任何直接或间接的计算算子数据依赖, 即不同核心在该阶段处理的计算任务在拓扑上完全独立;
2. **流水线负载统计与角色划分**: 统计波次 W_h 内各核心在各个实际出现的执行流水线上的有效计算工作量 $\text{work}(c)[p] = \sum_{v \in T_{h,c}, \text{pipe}(v)=p} \max(1, \text{cycles}_v)$, 确定该波次全系统工作量最大的主导流水线 p^* 。在主导流水线 p^* 上, 满足 $K \times \text{work}(c)[p^*] > \text{total}[p^*]$ (即本核工作量严格高于全波次平均工作量) 的核心被判定为**捐赠核心候选 (donor candidates)**; 满足 $K \times \text{work}(c)[p^*] < \text{total}[p^*]$ 且在波次 W_h 处分配有原有独立任务 T_{orig} 的核心被判定为**援助核心候选 (helper candidates)**。捐赠核心集与援助核心集基于主导流水线负载严格角色互斥, 在该波次完全空闲 (未分配任务) 的核心不进入援助核心集合;
3. **桥树 (Bridge-Tree) 分解与算子三相划分**: 在捐赠核心的原始任务 T_{donor} 内部, 通过 Tarjan 算法寻找无向图中的桥边, 并将桥边割开后的各个双边连通块压缩为节点, 构建**桥树 (bridge tree)** (而非基于割点的块割树)。若存在唯一的内部汇点算子, 且存在一个汇聚算子 $j \in T_{\text{donor}}$ 恰好接收两条来自独立前驱的桥边输入, 则将 T_{donor} 严格划分为三个互不相交的算子子集:
 - **导出独立支路 X** : 由其中一条前驱支路及其全部上游算子构成的独立连通块, 被选定为迁移至援助核心的子图;
 - **保留独立支路 Y** : 由另一条前驱支路、其余上游算子以及该 Task 内可能存在的其他独立连通分量构成, 继续保留在原捐赠核心上执行;
 - **汇合计算部分 J** : 由汇聚算子 j 及其在 T_{donor} 内部的全部后继后代算子构成, 继续保留在原捐赠核心上执行。

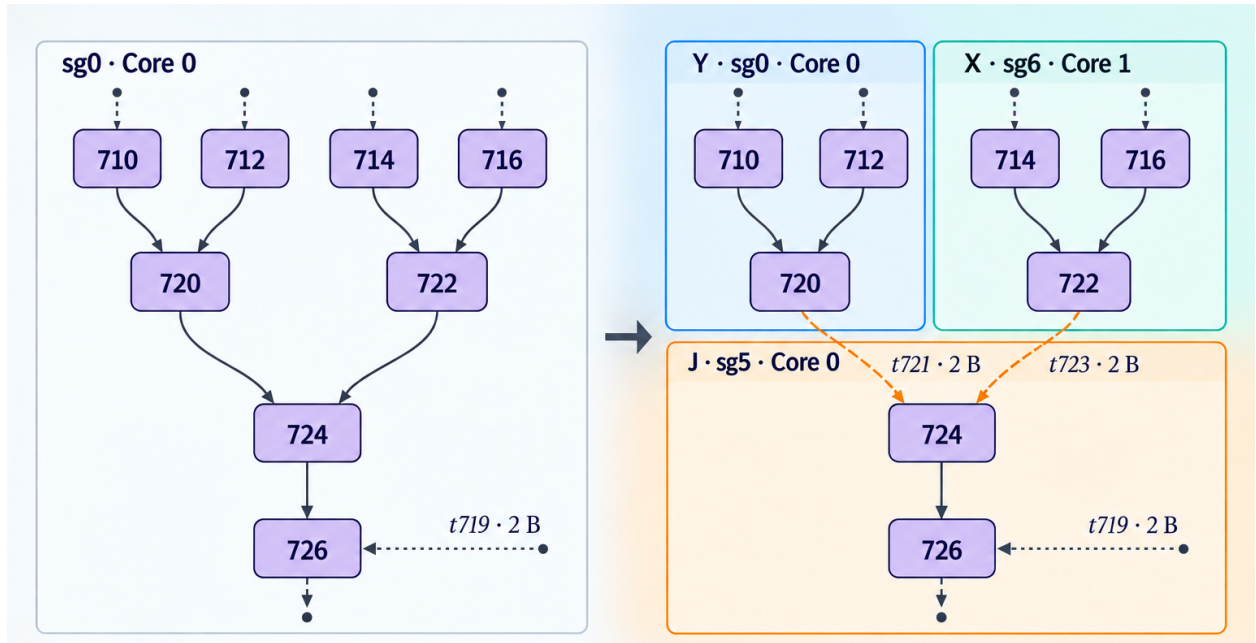


图 4.3-1 典型用例 case_026 (5 Cores) 独立分支重分配前后的切分结构映射。展示原始划分中 Core 0 上的超大任务解构为保留在 Core 0 上的独立支路 Y (算子 710、712、720) 与汇合计算 J (算子 724、726), 以及重分配至 Core 1 的导出独立支路 X (算子 714、716、722)。图中黑实线表示计算依赖, 橙虚线表示跨子图数据依赖, 点线端口表示连接至局部裁剪窗口外部的算子依赖 (非孤立无依赖节点)。图示仅对应特定用例局部的 8 节点子图结构, 不代表构造器处理的一般计算图全局结构。

算法 4.2 分支援助重分配的波次分支选择与分配算法

```
1 # 输入: 核心数 cores、波次 h、各核任务 by_core, 拆分记录 pieces 与任务
   数 n_tasks
2 # 输出: 该波次更新后的 pieces 与援助放置 placement (外围已做结构守卫)
3 loads, pstar = dominant_pipe_workloads(by_core, cores) # 各核工作量
   与主导流水线 p*
4 total = sum.loads[c][pstar] for c in range(cores)) # 包含未分配
   核心的零负载
5 donors = [c for c in by_core if cores * loads[c][pstar] > total]
6 helpers = [c for c in by_core if cores * loads[c][pstar] < total]
7 if not donors or not helpers:
8     return pieces, {}
9 found = find_exportable_branches(donors, by_core) # 桥树分析提取可导
   出支路 X/Y/J
10 if not found:
11     return pieces, {}
12
13 # 贪心指派 X 至增载后最大流水线工作量最小的 helper (独立拷贝避免负载失
   真)
14 new_loads = {c: Counter.loads[c]) for c in range(cores)}
15 placement = defaultdict(list)
16 for w in found:
17     new_loads[w["core"]].subtract(w["work"])
18     helper = min(helpers, key=lambda c: helper_load_key(new_loads, w
   , c))
19     new_loads[helper].update(w["work"])
20     placement[helper].append(w)
21 # 仅当全核最大流水线工作量严格下降且未超出构造器任务数限制时生效
22 if (max_load(new_loads) >= max_load.loads) or
23     n_tasks + 2 * (len(pieces) + len(found)) > max(320, n_tasks)
   ):
24     return pieces, {}
25 update_branch_pieces(pieces, found, placement, h, next_id) # 记录 X
   /Y/J 映射
26 return pieces, placement
```

源码对应: 834d8c9, src/q1/branch_aid.py::construct (第 359–438 行; 其中 dominant_pipe_workloads、find_exportable_branches、helper_load_key、max_load 与 update_branch_pieces 为伪代码抽象)。dominant_pipe_workloads 为未分配任务的核心补充零负载计数器, 并按全芯片总工作量最大且流水线 ID 升序破平确定 p^* ; helper_load_key 给出排序三元组 ($\max(\text{after.values}())$, $\sum(\text{after.values}())$, c); 外围假定单汇点与同高无跨核边, 仅在最大流水线工作量严格下降且满足构造器任务数限制 $\max(320, n_tasks)$ 时采纳重分配。

4.3.2 执行全序的构造

三相划分在算子依赖结构上严格满足:

- $X \cap Y = \emptyset$, $X \cap J = \emptyset$, $Y \cap J = \emptyset$, 且 $X \cup Y \cup J = T_{\text{donor}}$;
- X 与 Y 之间不存在任何正向或反向的数据依赖边;
- 所有跨子集依赖边均由 X 指向 J 或由 Y 指向 J ; J 绝不存在指向 X 或 Y 的反向反馈边;
- X 与 Y 的所有外部前驱算子均来自高度严格小于 h 的前驱波次任务。

在调度序列重构中, 算法支持一个援助核心在同一波次批量接收来自不同捐赠核心的多个独立导出支路 $X_1, X_2, \dots, X_m$:

- **捐赠核心的执行时序:** 原 Task T_{donor} 所在位置被替换为子图 Y 与子图 J 的顺次执行, 即在捐赠核心上 Y 紧邻排定在 J 之前 ($Y \rightarrow J$);
- **援助核心的执行时序:** 援助核心在执行其原有波次任务 T_{orig} 之前, 优先顺次排定执行其接收到的全部导出支路 $X_1, X_2, \dots, X_m$, 即援助核心上的执行序列构型为 $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_m \rightarrow T_{\text{orig}}$;
- **核心内波次推进时序:** 在每个核心的本地调度序列中, 任务严格按高度递增排列, 不同高度的任务顺序受本地排队序列与跨核数据依赖约束, 并非在全芯片施加全局物理同步屏障。

4.3.3 调度依赖图的无环性证明

命题 4.1 (分支援助重分配的无环性). 在上述波次推进、同波次独立性、三相依赖分解以及调度构建规则下, 重分配后的新方案 P_{new} 所诱导的扩展依赖图 $D^+(P_{\text{new}}) = (\mathcal{S}, E_{\mathcal{S}} \cup E_{\sigma})$ 必然不存在有向环。

证明. 根据命题 3.1, 只需构造一个严格单调的整数标号函数 $r: \mathcal{S} \rightarrow \mathbb{Z}$, 使得图中的每一条有向边 $(u, v) \in E_{\mathcal{S}} \cup E_{\sigma}$ 均满足 $r(u) < r(v)$ 。

对于任意子图 $s \in \mathcal{S}$, 设其在原始依赖图中所归属的波次高度为 $h(s)$ 。为每个波次 h 分配基准标号区间 $[100h, 100h + 99]$ 。在波次 h 内部, 针对不同角色子图定义整数标号如下:

1. 对于任意援助核心 $c \in H_h$, 设其接收到的导出支路序列为 $X_1, X_2, \dots, X_m$ 。由于每个捐赠核心至多导出一个分支 X 且捐赠核心与援助核心互斥, 必有 $m \leq |D_h| \leq K - 1 \leq 4 < 10$ 。其原有的波次任务记为 T_{orig} 。定义:

$$r(X_i) = 100h + 10 + i, \quad \forall i \in \{1, 2, \dots, m\}, \quad (4.4)$$

$$r(T_{\text{orig}}) = 100h + 30; \quad (4.5)$$

2. 对于任意捐赠核心 $c \in D_h$, 其分裂出的子图为 Y 与 J 。定义:

$$r(Y) = 100h + 10, \quad r(J) = 100h + 40; \quad (4.6)$$

3. 对于既非援助核心亦非捐赠核心的正常核心上的原有任务 T_{norm} , 定义:

$$r(T_{\text{norm}}) = 100h + 20. \quad (4.7)$$

现在逐一验证扩展依赖图 $D^+(P_{\text{new}})$ 中的所有可能边 (u, v) :

- **情形一: 跨波次边。**
 - 对于数据依赖边 $(u, v) \in E_{\mathcal{S}}$: 若 u 与 v 属于不同波次, 由于 Task 拓扑高度的定义, 所有外部前驱的高度严格小于后继任务的高度, 故必有 $h(u) \leq h(v) - 1$ 。因此 $r(u) \leq 100h(u) + 50 \leq 100(h(v) - 1) + 50 = 100h(v) - 50 < 100h(v) + 10 \leq r(v)$, 整数标号严格递增;

- 对于同一核心上的调度次序边 $(u, v) \in E_\sigma$ ：由于新方案严格保持波次单调推进，不同波次间的调度边同样满足 $h(u) < h(v)$ ，故必然有 $r(u) < r(v)$ 。
- 情形二：同波次内的核心调度边 $(u, v) \in E_\sigma$ 。
 - 在援助核心上，调度序列为 $X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_m \rightarrow T_{\text{orig}}$ 。根据标号赋值，显然有 $r(X_1) < r(X_2) < \dots < r(X_m) < r(T_{\text{orig}}) = 100h + 30$ ；
 - 在捐赠核心上，调度序列为 $Y \rightarrow J$ 。根据定义， $r(Y) = 100h + 10 < r(J) = 100h + 40$ ；
 - 其余核心在波次 h 内至多只有一个任务，不存在同波次调度边。
- 情形三：同波次内的数据依赖边 $(u, v) \in E_S$ 。
 - 根据同波次无依赖假设，波次 h 内不同原始任务之间不存在任何计算依赖边；
 - 唯一可能存在同波次数据依赖边的仅为由原任务分裂出的子图对。根据三相划分的依赖性质：
 - * 若边为 $X_i \rightarrow J$ ，则 $r(X_i) \leq 100h + 20 < 100h + 40 = r(J)$ ；
 - * 若边为 $Y \rightarrow J$ ，则 $r(Y) = 100h + 10 < 100h + 40 = r(J)$ ；
 - * X_i 与 Y 之间无依赖边； J 绝无指向 X_i 或 Y 的依赖边；
 - * 导出支路 X_i 与援助核心原有任务 T_{orig} 原本分属不同的独立任务，二者之间不存在任何数据依赖边。

综上所述，扩展依赖图 $D^+(P_{\text{new}})$ 中的任意有向边 $(u, v) \in E_S \cup E_\sigma$ 均恒成立 $r(u) < r(v)$ 。根据命题 3.1，图中绝不存在有向环，调度方案在拓扑执行时序上是严格死锁自由的。□

需要特别指出的是，命题 4.1 所证明的无环性构成了调度方案物理可执行的必要条件，但拓扑无环并不自动等价于片上存储资源不发生溢出。新增的子图切分改变了数据常驻生命周期，可能引发片上缓存动态容量紧张。因此，生成的候选方案必须在后续阶段由精确离散事件模拟器进行完整的容量验证与执行评分。

4.4 候选规模与采纳准则

为确保算法在复杂计算图上的执行可控性并避免不必要的评测开销，求解流程配备了严谨的资源保护与防退化机制：

- **任务数量工程规模约束 (TASK_LIMIT = 320)**：若父方案的任务总数已达到 320，直接跳过分支重分配；若新方案任务数超过 320，同样予以拒绝。该上限用于控制离散事件评测与后继编译的规模；
- **结构检查软时间预算 (EXTRA_SECONDS = 2.0s)**：在桥树分解与波次图遍历等深层图算法中设置 2.0 秒的软预算检测点。若遍历耗时超过该阈值，算法定期退出分析并安全抛出 `UnsupportedStructure` 异常回退至父级方案，避免在极端复杂图上陷入长时遍历；该软检测点不构成求解器端到端总耗时的硬性上限，外部离散评测耗时仍需另行累计；
- **单次评估预算受限**：分支重分配阶段至多允许发起 1 次额外的 E1 在线评测。全流程累计 E1 调用次数严格受限于 10 次；
- **严格字典序择优准则**：新方案以 $(M_1(P), \text{scheduled_copy_bytes})$ 为内部评价目标向量，仅在其严格呈字典序优于父方案时方予采纳。由于对于同一输入计算图，方案的调度复制总量与额外 DDR 搬运量之间满足恒等式 $\text{scheduled_copy_bytes} = \text{original_copy_bytes} + \text{extra_ddr_bytes}$ （其中原始输入输出搬运字节数 $\text{original_copy_bytes}$ 恒为确定常数），在固定输入图下该规则等价于本队按 $(M_1(P), \text{extra_ddr_bytes})$ 字典序择优的次序；此字典序平局规则为本队内部多候选比较选择策略，而非官方对多指标评价的硬性等价规定。

4.5 结果与分析

4.5.1 全量用例的性能比较

本文算法在官方提供的 100 张全量基准计算图、涵盖 1 至 5 核心的全部 500 种测试组合上进行了完整的端到端求解与评测，所有方案均顺利通过官方合法性校验并正常完成离散事件推进。测试结果依据 `paper/manuscript-v1/data/all-results.csv` 与固定哈希账本 (`source-receipt.json`, Commit `a1bb4451`) 进行了全量审计。

下表分别列出题目规定的平均加速比，以及使用算法输出方案计算的平均加速比；两行在一个核心时采用不同的方案。

表 4.1 问题一各核心数下的平均加速比与求解时间汇总

核心数量	1	2	3	4	5
官方口径平均加速比	1.0000	1.9516	2.7514	3.4786	4.0553
优化方案 $\text{mean}(B/M)$	1.0021	1.9516	2.7514	3.4786	4.0553
改善/持平/退化	0/100/0	8/92/0	15/85/0	12/88/0	13/87/0
求解时间均值 / s	0.436	3.316	3.426	3.427	3.522

如表中数据所示：

- 加速比表现：**在多核配置下，算法呈现出明确的并行加速收益（全量统计分布见图 4.5-1）。在 2 至 5 核场景下，官方口径下的平均加速比分别达到 1.9516、2.7514、3.4786 以及 4.0553。在 5 核配置下，相比前一固定版本（平均加速比 4.0259），当前方案取得了 0.729% 的相对性能增益；在单核场景下，算法输出方案相比官方单核基线同样取得了 1.0021 的轻微优化；
- 候选采纳与版本改善区分：**在全部 500 种测试组合中，分支援助重分配机制（branch-aid）在 36 个用例中成功触发并被择优采纳；而整套版本相比前一固定版本（v4）在多核心场景下共有 48 个用例取得改善（在 2、3、4、5 核下分别为 8、15、12、13 个，加总为 48 个用例），其余用例保持持平，全量评测无退化。需要指出的是，该 48 个用例的改善同时包含了前序结构细化与拓扑顺序调整等复合机制，目前尚未对分支援助重分配的独立贡献进行单独的消融实验隔离；
- 评测与调用账审计：**全量 500 个测试组合的求解过程累计发起 1130 次在线 E1 评测，单用例平均评测调用次数仅为 2.26 次，远低于 10 次的硬性预算；在主成绩的官方基准 E0 复验中，共执行了 48 次全新独立 E0 评测，其余 452 个用例在方案二进制文件、原始图、核心数及硬件配置逐项核同的前提下复用了历史已核结果；
- 通信开销与求解效率：**在 5 核心场景下，全量 100 个用例的额外 DDR 搬运量（`extra_ddr_bytes`）总和为 1,053,684,536 字节（较细化前基准小幅增加了 0.389%），调度复制总量（`scheduled_copy_bytes`）总和为 1,294,414,590 字节，客观呈现了细化方案在降低全局 Makespan 的同时成对伴随额外数据搬运量小幅增加的数值变化，此处仅记录成对方案指标对比的客观观测，不作未经独立消融隔离的因果机制归因。求解程序的端到端耗时表现为：采用最近秩次法（nearest-rank，即升序排列后取第 $\lceil 0.95n \rceil$ 项），全量 500 个测试组合的端到端平均求解时间为 2.825 秒，95% 分位数（第 475 位）为 13.164 秒，最大求解时间为 25.552 秒；若单独统计 5 核心的 100 个用例，平均求解时间为 3.522 秒，95% 分位数（第 95 位）为 16.623 秒，最大值为 25.552 秒。全部用例均大幅优于原题建议的 5~10 分钟运行区间。该测试在配备 4 进程并发的共享主机环境下完成，记录了真实的工程执行耗时。

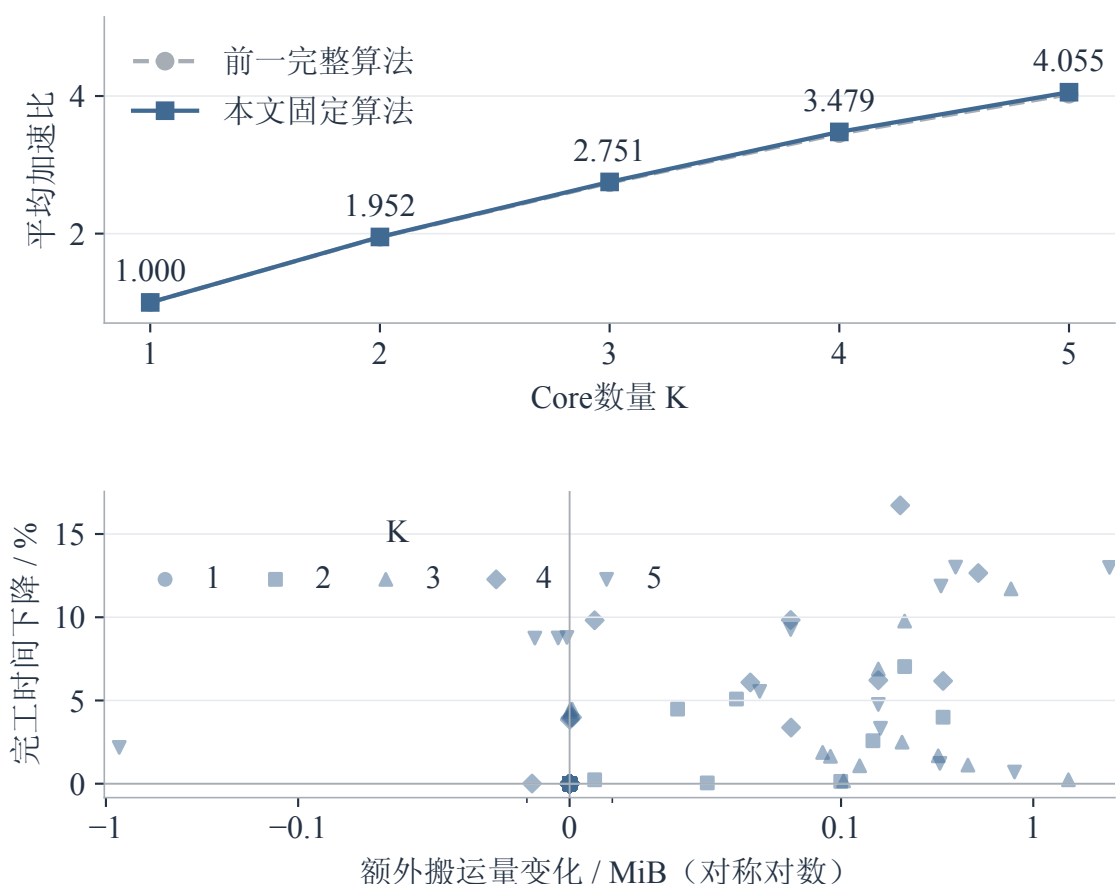


图 4.5-1 问题一的完整版本比较。上：每个核心数量的 100 图平均加速比；下：全部 500 种图与核心数量组合的时间与搬运量配对变化。核心数量用点形区分，横轴采用对称对数。

4.5.2 典型算例的执行时序分析

为了在微架构时钟周期与硬件流水线级别理解分支援助细化的执行特征，我们选取典型用例 `case_026` (5 Cores 配置) 进行结构与时序分析。

该用例在分支援助重分配前后的宏观指标对比体现如下：

- **细化前父方案：**由 `structural_refine` 生成 5 个 Task。官方 E0 仿真评测的 Makespan 为 44,114 cycles，调度产生的额外 DDR 搬运量为 211,680 字节；
- **细化后采纳方案：**经 `branch_aid` 调整后扩展为 7 个 Task。官方 E0 仿真评测的 Makespan 缩短为 42,014 cycles，调度产生的额外 DDR 搬运量为 375,532 字节。

方案调整使得该用例的全局完成时间降低了 2,100 cycles (完成时间降幅为 4.76%)，其代价是额外 DDR 数据搬运量增加了 163,852 字节。

图 4.5-2 绘制了基于官方模拟事件生成的执行时间线。从图中可以观察到微架构层面的负载分布与流水线事件：

1. **初始负载分析：**在原始划分中，波次 0 内部的算子集中在 Core 0 上的单个 Task 中，其在向量流水线上的预估计算工作量代理峰值为 40,824 cycles；
2. **分支剥离与工作量调整：**算法识别出 Task 内部由算子 714、716、722 组成的独立向量分支 X，并将其剥离为子图 6 迁移至 Core 1 上执行；原 Core 0 则保留分支 Y (子图 0, 包含算子 710、712、720) 与汇合部分 J (子图 5, 包含算子 724、726)。在调度序列

中，Core 1 排定优先执行支路 X，各核心各流水线的预估计算工作量代理峰值由 40,824 cycles 降至 37,569 cycles；

3. **完成时间表现：**跨核迁移使得算子 722 生成的中间张量需要通过 DDR 读写传递给 Core 0 上的汇合算子 724，额外 DDR 搬运量增加了 163,852 字节。在官方模拟推进下，方案整体 Makespan 呈现出 2,100 cycles 的净缩减；由于微架构事件中并发计算、访存排队与同步等待相互交织，此处 2,100 cycles 的改善反映了方案调整后的整体模拟结果，而不宜在缺乏消融隔离的情况下直接归因于单一维度的因果分摊。

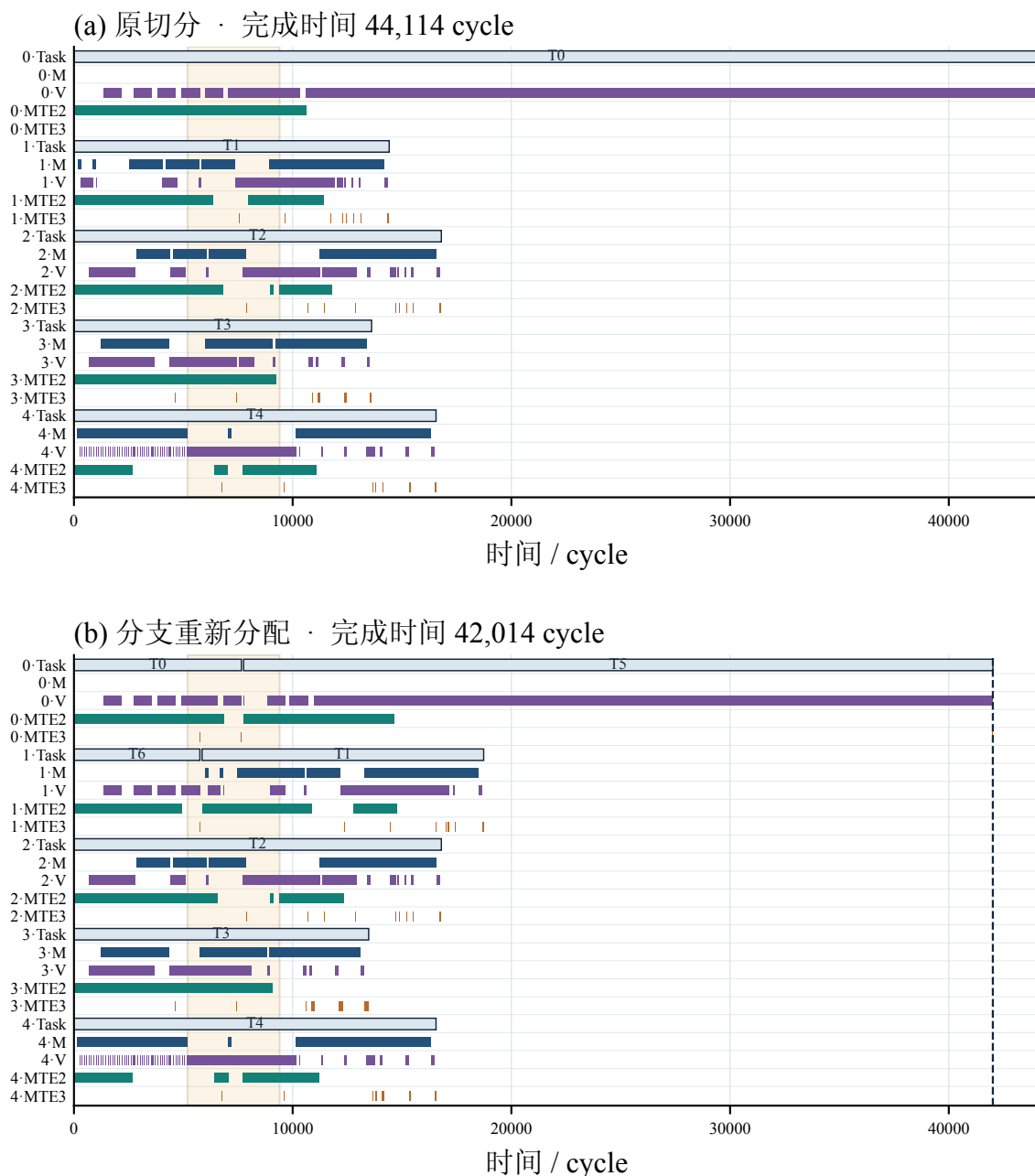


图 4.5-2 用例 case_026 在 5 核配置下重分配前后的完整执行时间线对比。基于官方模拟事件绘制，(a) 原始方案 Makespan 为 44,114 cycles，额外数据搬运量为 211,680 B；(b) 分支援助重分配后 Makespan 为 42,014 cycles，额外数据搬运量为 375,532 B。各核心 Task 行表示任务执行区间，M、V、MTE2、MTE3 分别表示四类硬件流水线的已记录操作区间，图中浅色区域标出 5,200–9,400 cycles 局部观察窗口。

在局部放大图（图 4.5-3）中，展示了 5,200–9,400 cycles 关键时间窗内核心 0 与核心 1 的流水线推进细节：图内 C0:T0/SG0 表示核心 0 上执行的任务 0（对应子图 0，即保留分支 Y）；重分配后在核心 1 上优先执行的独立分支 X（包含算子 714、716、722），其在向量流水线上密集执行，随后核心 1 推进执行其原有任务 C1:T1/SG1；核心 0 上则在完成保留支路 Y 后，紧接着执行汇合任务 C0:T5/SG5（子图 5，对应 J）。在两组时间线中，执行资源对应的空白仅表示该时段没有记录到流水线占用事件，不能据此主观判定其原因是等待外部内存、片上存储或其他硬件资源。

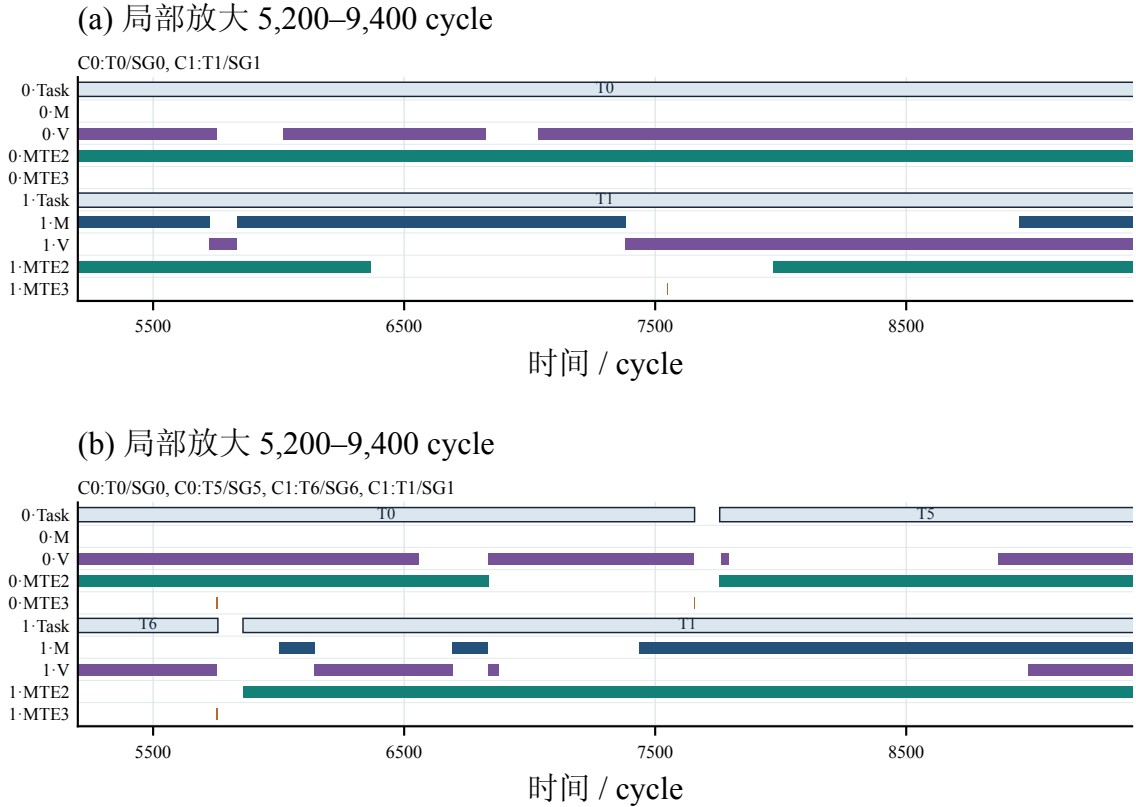


图 4.5-3 用例 case_026 在 5,200–9,400 cycles 窗口内的局部微架构流水线时序放大图（对应图 4.5-2 浅色区域）。展示核心 0 与核心 1 在该时段内的任务切换与各流水线操作事件分布。图中执行资源对应的空白仅表示该时段未记录到流水线占用事件，不推定具体的外部等待原因。

该案例反映了多核心硬件调度中的典型权衡：通信量的局部上升并不必然导致整体性能劣化；当多核心并行推进缩短排队的效果超过数据搬运与同步时延时，即可在官方模拟中获得全局完成时间的改善。

4.6 本章小结

本章针对跨任务数据必须经由外部主存（DDR）中转且存在跨核同步等待的执行约束，利用连通分量与拓扑前沿生成基础划分，并通过桥树识别独立计算分支实施分支援助重分配（Branch-Aid），按拓扑波次保障无环。在 100 张基准图与 1 至 5 核心（500 种测试组合）上，5 核心平均加速比达 4.0553。完整版本在缩短执行时间的同时伴随部分搬运量增加，不作单一机制的独立因果归因。

5 问题二：数据复用下的多核调度

5.1 问题分析与数据复用约束

在问题二的系统设定中，被分配到同一个计算核心的全部子图在底层编译时被合并为一个整体硬件执行任务（Task）。这一微架构语义明确了同核内的数据流动特征：在同一核心内部，先前执行的子图算子所生成的数据张量可以直接保留在该核心的片上私有缓存（L1 Buffer 与 Unified Buffer, UB）中，供后继同核算子直接读取复用，无需再经由外部主存（DDR）进行持久化中转；而跨核心数据传递则依然必须经由外部主存通过跨核通信链路进行中转。

若数据生成算子与数据消费算子被分配至不同的计算核心，则数据依然必须通过跨核通信链路传递：由源核心执行 COPY_OUT 将数据写入 DDR，再由目标核心执行 COPY_IN 将数据读入本地缓存。在硬件时序上，目标核心的读取操作必须在源核心写出操作彻底完成 500 cycles 之后方可启动。在此执行模型下，即使在方案提交文件（node_to_subgraph）中将每个计算算子单独划分为一个子图，也不会在硬件底层为每个算子生成孤立的物理 Task，子图序列仅用于指示底层编译器在各核心内部排定算子的先后执行次序。

同核心任务合并加深了算子分配与调度顺序之间的耦合关联，并引出了新的微架构存储权衡：

1. **缓存复用收益：**当某一数据张量的生成算子与全部消费算子均被分配至同一计算核心、且不存在其他跨核消费者时，可免去该张量对应的跨核 DDR 读写搬运与 500 cycles 的跨核同步等待；若仍有部分消费者位于远端核心或存在外部主存写出，则相应的边界传输依然发生；
2. **片上存储驻留压力与溢出代价：**若过度追求局部复用而将算子集中，或者消费同一张量的多个算子在执行时序上相距过远，该中间张量就必须在片上私有缓存中维持长时间的驻留生命周期。当片上缓存容量不足以容纳活跃数据峰值时，官方编译系统将在其内部流程的第二阶段（Step 2）强制插入溢出换出与换入操作（spill），从而在执行阶段产生意料之外的二次 DDR 读写开销；
3. **空间复用与时间并行的冲突：**将数据消费算子分散至其他核心虽会引入边界数据搬运，但能有效利用多核计算资源展开并行计算，并释放本地片上存储压力。

因此，私有缓存容量限制与 Step 2 溢出（spill）深刻影响着系统的实际数据搬运总量。多核心调度算法必须在控制跨核数据流动的同时，防止片上工作量与存储过度聚集。

图 5.1-1 采用一个包含 9 个物理张量的手算结构用例，展示了单核心排队执行时片上私有缓存占用的动态变化过程。需要指出的是，该图展示的是单核心在静态编译序列下的结构示意，横轴展示操作序号 $i = 0 \dots 7$ （其中 0 与 7 分别对应初始与全部释放的空占用边界；该序号为模仿官方编译第二阶段 Step 2 静态操作顺序的手算示例，展示顺序编译次序而非微架构物理并行执行周期 cycles），并非全量 100 张基准计算图的实测记录或真实的并行执行时间线。每个物理张量 t 在该核心的生命周期定义为闭区间 $[f(t), \ell(t)]$ ，分别对应应在该静态编译序列中第一次与最后一次触碰该物理张量的操作序号（末次触碰包含计算读取或外存写出等搬运操作）；张量在末次触及步骤 $\ell(t)$ 期间仍持续驻留在片上私有缓存中，直至后继步骤启动时方被完全释放。具体而言，张量 T_6 的生命周期区间为 $[5, 6]$ ，张量 U_3 为 $[4, 6]$ 。在该手算用例中，容量曲线未超出配置上限属于基于静态步数累加的手算事实；由于本示例并未运行官方离散事件仿真，图中无官方 spill 事件记录且未绘制溢出事件。在单核心物理容量（L1 Buffer 容量为 512 KiB，Unified Buffer 容量为 128 KiB）约束下，若某步骤同时存在多项未释放的输入张量与新分配的输出张量，将产生瞬时存储占用峰值（图中示例在步骤 2 达到 L1 峰值 512 KiB，在步骤 4 达到 UB 峰值 72 KiB）。该示例为理解调整算子顺序能够规避潜在溢出换出提供了物理直观；本文主算法并未在此建立全局无溢出的精确容量调度保证。在实际求解中，算法采用分层控制机制：超图割目标 D_{pre2}